

Rust 国际象棋引擎 — 详细开发设计文档

概述

本文档描述一个现代化、模块化、高性能的 Rust 国际象棋引擎的完整架构设计。设计参考 Stockfish 等现代棋类引擎，充分利用 Rust 类型系统和零成本抽象，以 Bitboard 为核心数据结构。

一、项目架构总览

1.1 Workspace 结构

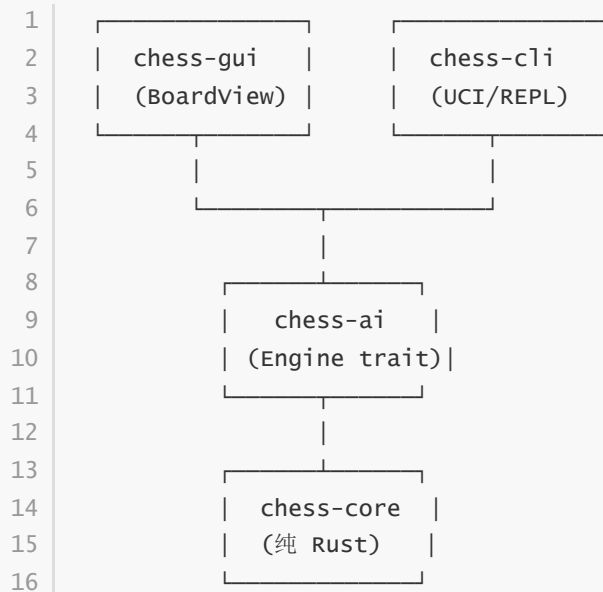
1	chess-engine/	
2	├─ Cargo.toml	# workspace 根配置
3	├─ README.md	
4	├─ DESIGN.md	# 本文档
5	├─ rust-toolchain.toml	# 固定工具链版本
6	├─ .rustfmt.toml	# 格式化配置
7	├─ clippy.toml	# clippy 配置
8		
9	├─ chess-core/	# 核心引擎 crate
10	├─ Cargo.toml	
11	└─ src/	
12	├─ lib.rs	# crate 根, re-export 所有公开类型
13	├─ square.rs	# Square 类型 (0-63)
14	├─ color.rs	# Color 枚举 (white/black)
15	├─ piece.rs	# PieceKind + Piece 组合类型
16	├─ bitboard.rs	# BitBoard(u64) + 运算
17	├─ board.rs	# 12 bitboard 棋盘表示
18	├─ position.rs	# 完整棋局状态
19	├─ mv.rs	# 压缩 Move(u32)
20	├─ movegen/	# 走法生成模块
21	├─ mod.rs	# MoveGenerator trait + 调度
22	├─ pawn.rs	# 兵走法生成
23	├─ knight.rs	# 马走法生成
24	├─ bishop.rs	# 象走法生成
25	├─ rook.rs	# 车走法生成
26	├─ queen.rs	# 后走法生成
27	└─ king.rs	# 王走法生成
28	├─ attack.rs	# 攻击表 + is_attacked
29	├─ legality.rs	# 走法合法性检查
30	├─ makemove.rs	# MoveMaker: apply + undo
31	├─ zobrist.rs	# Zobrist 哈希
32	├─ game.rs	# Game 高级 API
33	├─ fen.rs	# FEN 解析与输出
34	├─ castling.rs	# 王车易位权限
35	├─ perft.rs	# Perft 测试工具
36	└─ error.rs	# 错误类型
37		
38	├─ chess-ai/	# AI 引擎 crate
39	├─ Cargo.toml	
40	└─ src/	
41	└─ lib.rs	

```

42 | | | engine.rs          # ChessEngine trait
43 | | | random.rs         # RandomEngine
44 | | | alphabeta.rs      # AlphaBetaEngine
45 | | | eval.rs          # 局面评估
46 | | | search.rs        # 搜索框架
47 | |
48 | | chess-cli/          # CLI 界面 crate
49 | | | Cargo.toml
50 | | | src/
51 | | | | main.rs        # 入口
52 | | | | uci.rs         # UCI 协议
53 | | | | repl.rs       # 交互式 REPL
54 | |
55 | | chess-gui/          # GUI crate (后续)
56 | | | Cargo.toml
57 | | | src/
58 | | | | lib.rs
59 | | | | view.rs        # BoardView trait
60 | |
61 | | benches/            # 性能基准测试
62 | | | Cargo.toml
63 | | | src/
64 | | | | bench.rs

```

1.2 依赖关系图



规则:

- `chess-core` 零外部依赖 (仅使用 `std`, 可选 `bitflags`)
- `chess-ai` 仅依赖 `chess-core`
- `chess-gui` / `chess-cli` 依赖 `chess-core` + `chess-ai`
- 任何 crate 不得绕过 `chess-core` 的公共 API 直接操作内部状态

1.3 crate 类型与版本

Crate	类型	Rust Edition	主要依赖
chess-core	lib	2024	<code>bitflags</code> (可选), <code>arrayvec</code>
chess-ai	lib	2024	<code>chess-core</code> , <code>rand</code>
chess-cli	bin	2024	<code>chess-core</code> , <code>chess-ai</code> , <code>clap</code>
chess-gui	lib	2024	<code>chess-core</code>
benches	bench	2024	<code>chess-core</code> , <code>chess-ai</code> , <code>criterion</code>

二、chess-core 详细设计

2.1 Square (棋盘格)

```
1  #[derive(Copy, Clone, Eq, PartialEq, Hash, Debug)]
2  pub struct Square(u8);
```

设计要点:

- 内部 `u8`, 范围 0-63, 对应 a1=0, b1=1, ..., h8=63 (Little-Endian Rank-File mapping)
- 构造函数 `Square::new(index: u32) -> Option<Square>` —— 返回 `Option`, 非法值返回 `None`
- 提供 `unsafe Square::new_unchecked(index: u32) -> Square` 用于性能关键路径 (需证明安全性)
- 方法:
 - `index(&self) -> usize` —— 数组索引
 - `bit(&self) -> u64` —— 返回 `1u64 << self.0`
 - `rank(&self) -> u8` —— 0-7 (rank 1 = 0)
 - `file(&self) -> u8` —— 0-7 (a file = 0)
 - `from_coord(file: u8, rank: u8) -> Option<Square>` —— 从坐标构造
 - `Display` trait —— 输出 "e4" 这样的代数表示

常量: 提供 `Square::A1 ~ Square::H8` 共 64 个命名常量。

2.2 Color (颜色)

```
1  #[derive(Copy, Clone, Eq, PartialEq, Hash, Debug)]
2  #[repr(u8)]
3  pub enum Color {
4      white = 0,
5      Black = 1,
6  }
```

- `#[repr(u8)]` 保证可以安全 transmute 为 `usize` 用于数组索引

- 提供 `flip(&self) -> Color` —— 切换颜色
- 提供 `From<Color> for usize` 实现

2.3 Piece (棋子)

```
1  #[derive(Copy, Clone, Eq, PartialEq, Hash, Debug)]
2  #[repr(u8)]
3  pub enum PieceKind {
4      Pawn = 0,
5      Knight = 1,
6      Bishop = 2,
7      Rook = 3,
8      Queen = 4,
9      King = 5,
10 }
11
12 #[derive(Copy, Clone, Eq, PartialEq, Hash, Debug)]
13 pub struct Piece {
14     pub color: Color,
15     pub kind: PieceKind,
16 }
```

- `PieceKind` 提供 `const COUNT: usize = 6`
- `Piece` 提供 `to_char(&self) -> char` (如白兵 = 'P', 黑后 = 'q')
- 提供 `Piece::new(color: Color, kind: PieceKind) -> Piece`
- `From<PieceKind> for usize` 和 `From<Color> for usize` 实现

2.4 BitBoard (位棋盘)

```
1  #[derive(Copy, Clone, Eq, PartialEq, Hash, Debug)]
2  pub struct BitBoard(u64);
```

核心方法:

方法	签名	说明
<code>empty()</code>	<code>-> Self</code>	返回空位棋盘
<code>full()</code>	<code>-> Self</code>	返回全满位棋盘
<code>set(&mut self, sq: Square)</code>		设置一位
<code>clear(&mut self, sq: Square)</code>		清除一位
<code>contains(&self, sq: Square) -> bool</code>		测试一位
<code>is_empty(&self) -> bool</code>		是否为空
<code>pop_count(&self) -> u32</code>		1 的个数 (使用 <code>count_ones()</code>)
<code>lsb(&self) -> Option<Square></code>		最低有效位 (bitscan forward)

方法	签名	说明
<code>pop_lsb(&mut self) -> Option<Square></code>		弹出最低有效位
<code>as_u64(&self) -> u64</code>		获取原始值

Trait 实现:

```

1  impl BitOr for BitBoard { type Output = Self; } // 并集
2  impl BitAnd for BitBoard { type Output = Self; } // 交集
3  impl Not for BitBoard { type Output = Self; } // 补集
4  impl BitOrAssign for BitBoard { } // |=
5  impl BitAndAssign for BitBoard { } // &=
6  impl BitXor for BitBoard { type Output = Self; } // 异或

```

迭代器:

```

1  pub struct BitBoardIter {
2      bb: BitBoard,
3  }
4
5  impl Iterator for BitBoardIter {
6      type Item = Square;
7      // 每次 pop_lsb, 返回 Square
8  }
9
10 impl IntoIterator for BitBoard {
11     type Item = Square;
12     type IntoIter = BitBoardIter;
13 }

```

这样支持 `for sq in bitboard { ... }` 语法。

位运算优化: `pop_lsb()` 使用 `bb.0.trailing_zeros()` 和 `bb.0 &= bb.0 - 1` (Brian Kernighan 算法)。

2.5 Board (棋盘)

```

1  pub struct Board {
2      pieces: [[BitBoard; 6]; 2], // [color][piece_kind]
3      by_square: [Option<Piece>; 64], // 快速反向查找 (cache-friendly)
4  }

```

设计理由:

- `pieces[color][kind]` 是主要的位棋盘存储, 12 个 BitBoard
- `by_square` 提供 O(1) 的 `piece_at(sq)` 查询, 避免遍历 12 个位棋盘
- 两个数据结构保持同步更新

核心方法:

```

1  impl Board {
2      pub fn new() -> Self;
3      pub fn piece_at(&self, sq: Square) -> Option<Piece>;
4      pub fn pieces(&self, color: Color) -> BitBoard;           // 某颜色所有棋子
5      pub fn pieces_kind(&self, color: Color, kind: PieceKind) -> BitBoard;
6      pub fn occupied(&self) -> BitBoard;                       // 所有棋子
7      pub fn occupied_by(&self, color: Color) -> BitBoard;     // 某颜色棋子
8      pub fn empty(&self) -> BitBoard;                          // 空格
9      pub fn add_piece(&mut self, sq: Square, piece: Piece);
10     pub fn remove_piece(&mut self, sq: Square) -> Option<Piece>;
11     pub fn move_piece(&mut self, from: Square, to: Square) ->
12         Option<Piece>;
13     pub fn king_square(&self, color: Color) -> Square;       // 王的位置
14 }

```

不变量: `by_square` 与 `pieces` 位棋盘始终一致。

2.6 CastlingRights (王车易位权限)

```

1  use bitflags::bitflags;
2
3  bitflags! {
4      #[derive(Copy, Clone, Eq, PartialEq, Hash, Debug)]
5      pub struct CastlingRights: u8 {
6          const NONE = 0;
7          const WHITE_KING_SIDE = 1 << 0;
8          const WHITE_QUEEN_SIDE = 1 << 1;
9          const BLACK_KING_SIDE = 1 << 2;
10         const BLACK_QUEEN_SIDE = 1 << 3;
11         const ALL = WHITE_KING_SIDE | WHITE_QUEEN_SIDE
12             | BLACK_KING_SIDE | BLACK_QUEEN_SIDE;
13     }
14 }

```

相关常量 (关联到 Square) :

- 王车易位涉及的关键格: 初始王/车位置、经过格、目标格
- 掩码: 王翼空位掩码、后翼空位掩码、攻击检测掩码

2.7 Move (走法)

```

1  #[derive(Copy, Clone, Eq, PartialEq, Hash, Debug)]
2  pub struct Move(u32);

```

位布局:

```

1  bits 0-5:   from square (6 bits, 0-63)
2  bits 6-11:  to square   (6 bits, 0-63)
3  bits 12-15: promotion   (4 bits, PieceKind)
4  bits 16-19: flags       (4 bits, MoveFlag)
5  bits 20-31: reserved    (12 bits, for future: score, etc.)

```

MoveFlag 枚举:

```

1  #[derive(Copy, Clone, Eq, PartialEq, Debug)]
2  #[repr(u8)]
3  pub enum MoveFlag {
4      Quiet = 0,
5      DoublePawnPush = 1,
6      KingCastle = 2,
7      QueenCastle = 3,
8      Capture = 4,
9      EnPassant = 5,
10     Promotion = 8,          // bit 3 set = promotion
11     // Promotion + piece kind
12     KnightPromotion = 8,    // 1000
13     BishopPromotion = 9,    // 1001
14     RookPromotion = 10,     // 1010
15     QueenPromotion = 11,    // 1011
16     // Promotion capture variants
17     KnightPromotionCapture = 12, // 1100
18     BishopPromotionCapture = 13, // 1101
19     RookPromotionCapture = 14,   // 1110
20     QueenPromotionCapture = 15,  // 1111
21 }

```

Move 方法:

```

1  impl Move {
2      pub fn new(from: Square, to: Square, flag: MoveFlag) -> Self;
3      pub fn new_promotion(from: Square, to: Square, kind: PieceKind,
4      capture: bool) -> Self;
5      pub const NULL: Self; // 空走法 (0)
6      pub fn from(&self) -> Square;
7      pub fn to(&self) -> Square;
8      pub fn flag(&self) -> MoveFlag;
9      pub fn promotion(&self) -> Option<PieceKind>;
10     pub fn is_capture(&self) -> bool;
11     pub fn is_promotion(&self) -> bool;
12     pub fn is_castle(&self) -> bool;
13     pub fn is_quiet(&self) -> bool;
14 }

```

2.8 Position (棋局状态)

```

1  pub struct Position {
2      board: Board,
3      side_to_move: Color,
4      castling: CastlingRights,
5      en_passant: Option<Square>,
6      halfmove_clock: u32,    // 50步规则计数器
7      fullmove_number: u32,   // 从1开始
8      zobrist_key: u64,       // Zobrist 哈希 (增量更新)
9      // 可选: 局面历史哈希 (用于三次重复检测)
10 }

```

公开 API (只读访问) :

```

1  impl Position {
2      pub fn board(&self) -> &Board;
3      pub fn side_to_move(&self) -> Color;
4      pub fn castling(&self) -> CastlingRights;
5      pub fn en_passant(&self) -> Option<Square>;
6      pub fn halfmove_clock(&self) -> u32;
7      pub fn fullmove_number(&self) -> u32;
8      pub fn zobrist_key(&self) -> u64;
9      pub fn piece_at(&self, sq: Square) -> Option<Piece>;
10 }

```

FEN 支持:

- `Position::from_fen(fen: &str) -> Result<Position, ChessError>` —— 解析 FEN
- `Position::to_fen(&self) -> String` —— 输出 FEN
- 支持标准起始局面: `startpos()` 快捷方法

2.9 攻击表模块 (attack.rs)

预计算查找表:

```

1  // 64 个元素，每个预先计算好
2  pub static KNIGHT_ATTACKS: [BitBoard; 64] = /* 编译期/启动时初始化 */;
3  pub static KING_ATTACKS: [BitBoard; 64] = /* ... */;
4  pub static PAWN_ATTACKS: [[BitBoard; 64]; 2] = /* [color][square] */;

```

滑动棋子射线:

```

1  // 使用 const fn 或 lazy_static/OnceLock 初始化
2  pub fn bishop_rays(sq: Square, occupied: BitBoard) -> BitBoard;
3  pub fn rook_rays(sq: Square, occupied: BitBoard) -> BitBoard;
4  pub fn queen_rays(sq: Square, occupied: BitBoard) -> BitBoard {
5      bishop_rays(sq, occupied) | rook_rays(sq, occupied)
6  }

```

射线计算算法 (Phase 3 实现) :

- 经典方法: 对 4 个方向 (主教) / 4 个方向 (车) , 使用 `trailing_zeros` 扫描到第一个阻挡
- Phase 8 优化: 升级为 Magic Bitboard

核心检查函数:

```

1  /// 判断 sq 是否被 attack_color 颜色的棋子攻击
2  pub fn is_square_attacked(
3      board: &Board,
4      sq: Square,
5      by_color: Color,
6  ) -> bool;

```

检测顺序 (从便宜到贵) :

1. 兵攻击 (查 PAWN_ATTACKS 表)

- 2. 马攻击 (查 KNIGHT_ATTACKS 表)
- 3. 王攻击 (查 KING_ATTACKS 表)
- 4. 滑动棋子 (计算射线, 检查对应棋子存在性)

2.10 走法生成模块 (movegen/)

MoveGenerator trait:

```
1 pub trait MoveGenerator {
2     /// 生成指定局面的所有伪合法走法
3     fn generate(&self, position: &Position) -> Vec<Move>;
4 }
```

各棋子生成器设计:

模块	核心算法
pawn.rs	查找表 + 位运算: 前推、双推、攻击 (吃子+过路兵)、升变
knight.rs	查 KNIGHT_ATTACKS 表, 过滤己方棋子
bishop.rs	计算主教射线, 过滤己方棋子
rook.rs	计算车射线, 过滤己方棋子
queen.rs	主教射线 U 车射线
king.rs	查 KING_ATTACKS 表 + 王车易位生成

聚合生成器:

```
1 pub struct LegalMoveGenerator;
2
3 impl LegalMoveGenerator {
4     /// 生成所有合法走法 (伪合法 + 合法性过滤)
5     pub fn generate_legal(position: &Position) -> Vec<Move>;
6
7     /// 生成所有伪合法走法
8     pub fn generate_pseudo_legal(position: &Position) -> Vec<Move>;
9 }
```

性能设计:

- 使用栈分配的 `ArrayVec<Move, 256>` 避免堆分配 (最大合法走法数约 218)
- 攻击检测短路: 先检测简单攻击 (兵、马), 再检测滑动棋子
- 王车易位合法性: 先检查空位, 再检查攻击

2.11 走法执行模块 (makemove.rs)

Undo 结构:

```
1 pub struct Undo {
2     mv: Move,
3     captured: Option<Piece>,
4     prev_castling: CastlingRights,
5     prev_en_passant: Option<Square>,
6     prev_halfmove: u32,
7     prev_zobrist: u64,
8 }
```

MoveMaker:

```
1 pub struct MoveMaker;
2
3 impl MoveMaker {
4     /// 执行走法, 返回 Undo 信息
5     pub fn make_move(position: &mut Position, mv: Move) -> Undo;
6
7     /// 撤销走法
8     pub fn unmake_move(position: &mut Position, undo: Undo);
9 }
```

make_move 执行流程:

```
1 1. 保存 Undo 信息
2 2. 清除 en_passant (除非本次产生新的)
3 3. 根据 MoveFlag 分发:
4   - Quiet: 简单移动棋子
5   - DoublePawnPush: 移动棋子 + 设置 en_passant
6   - KingCastle: 移动王 e->g + 移动车 h->f
7   - QueenCastle: 移动王 e->c + 移动车 a->d
8   - Capture: 移除目标格棋子 + 移动棋子
9   - EnPassant: 移除过路兵 + 移动兵
10  - Promotion*: 移除目标格棋子(如有) + 替换升变棋子
11 4. 更新 CastlingRights (如涉及王或车移动)
12 5. 更新 halfmove_clock / fullmove_number
13 6. 增量更新 zobrist_key
14 7. 切换 side_to_move
```

unmake_move 执行流程: 完全逆向以上步骤, 使用 Undo 恢复状态。

2.12 Zobrist 哈希 (zobrist.rs)

```
1 pub struct Zobrist;
2
3 impl Zobrist {
4     /// 初始化所有随机数 (使用确定性种子或固定随机数)
5     pub fn init();
6
7     /// 棋子键值
```

```

8     pub fn piece_key(piece: Piece, sq: Square) -> u64;
9
10    /// 王车易位键值
11    pub fn castling_key(castling: CastlingRights) -> u64;
12
13    /// 过路兵键值
14    pub fn en_passant_key(sq: Option<Square>) -> u64;
15
16    /// 走子方键值（黑方回合的 XOR 常数）
17    pub fn side_key() -> u64;
18 }

```

增量更新：Position 的 `zobrist_key` 在 `make_move` / `unmake_move` 中增量更新，避免每次重新计算。

2.13 Game API (game.rs)

```

1  pub struct Game {
2      position: Position,
3      history: Vec<(Move, Undo)>,
4  }
5
6  impl Game {
7      /// 从标准起始局面创建
8      pub fn new() -> Self;
9
10     /// 从 FEN 创建
11     pub fn from_fen(fen: &str) -> Result<Self, ChessError>;
12
13     /// 获取当前局面（只读引用）
14     pub fn position(&self) -> &Position;
15
16     /// 获取当前所有合法走法
17     pub fn legal_moves(&self) -> Vec<Move>;
18
19     /// 执行走法
20     pub fn play(&mut self, mv: Move) -> Result<(), ChessError>;
21
22     /// 撤销上一步走法
23     pub fn undo(&mut self) -> Result<(), ChessError>;
24
25     /// 检测游戏是否结束
26     pub fn is_game_over(&self) -> bool;
27
28     /// 检测是否将军
29     pub fn is_check(&self) -> bool;
30
31     /// 检测是否将杀
32     pub fn is_checkmate(&self) -> bool;
33
34     /// 检测是否逼和
35     pub fn is_stalemate(&self) -> bool;
36
37     /// 走法历史
38     pub fn history(&self) -> &[(Move, Undo)];
39

```

```

40     /// 重置
41     pub fn reset(&mut self);
42 }

```

设计原则： GUI 和 AI 只能通过 `Game` 类型操作棋局，无法访问内部 `Bitboard`。

2.14 错误类型 (error.rs)

```

1  #[derive(Debug, Clone, Eq, PartialEq)]
2  pub enum ChessError {
3      InvalidSquare(u32),
4      InvalidFen(String),
5      InvalidMove(Move),
6      NoKing(Color),
7      GameOver,
8      NothingToUndo,
9      InvalidPromotion,
10     ParseError(String),
11 }
12
13 impl std::fmt::Display for ChessError { ... }
14 impl std::error::Error for ChessError { ... }

```

三、chess-ai 设计

3.1 ChessEngine trait

```

1  pub trait ChessEngine {
2      /// 搜索并返回最佳走法
3      fn search(&mut self, position: &Position) -> Option<Move>;
4
5      /// 引擎名称
6      fn name(&self) -> &str;
7
8      /// 设置搜索时间限制（毫秒）
9      fn set_time_limit(&mut self, ms: u64);
10
11     /// 设置搜索深度限制
12     fn set_depth_limit(&mut self, depth: u32);
13 }

```

3.2 RandomEngine

- 从合法走法中随机选择
- 用于测试和基准对比
- 实现 `ChessEngine` trait

3.3 AlphaBetaEngine

```
1  pub struct AlphaBetaEngine {
2      max_depth: u32,
3      time_limit_ms: Option<u64>,
4      nodes_searched: u64,
5      transposition_table: TranspositionTable, // Phase 8
6  }
7
8  impl AlphaBetaEngine {
9      pub fn new(max_depth: u32) -> Self;
10
11     fn alpha_beta(
12         &mut self,
13         position: &mut Position,
14         depth: u32,
15         alpha: i32,
16         beta: i32,
17     ) -> i32;
18
19     fn quiescence_search(
20         &mut self,
21         position: &mut Position,
22         alpha: i32,
23         beta: i32,
24     ) -> i32;
25
26     fn evaluate(&self, position: &Position) -> i32;
27 }
```

搜索优化 (Phase 8) :

- 走法排序 (MVV-LVA)
- 迭代加深
- 置换表
- 空着裁剪 (Null Move Pruning)
- 杀手走法 (Killer Moves)
- 历史启发 (History Heuristic)

3.4 评估函数 (eval.rs)

Phase 7 基本评估:

```
1 | Score = Material + PieceSquareTable + Mobility
```

- **子力价值**: P=100, N=320, B=330, R=500, Q=900, K=20000
- **棋子位置表** (Piece-Square Tables) : 从白方视角, 黑方镜像
- **基本机动性**: 合法走法数量的少量加分

Phase 8 扩展:

- 兵结构 (孤兵、叠兵、通路兵)

- 王安全
- 为 NNUE 预留输入接口

四、chess-cli 设计

4.1 命令行接口

```
1  chess-cli 0.1.0
2  Rust Chess Engine
3
4  USAGE:
5      chess-cli [OPTIONS] [COMMAND]
6
7  COMMANDS:
8      play      启动交互式对局
9      perft     运行 perft 测试
10     bench     运行基准测试
11     uci       启动 UCI 模式
12     fen <FEN> 从 FEN 加载并分析局面
13
14  OPTIONS:
15      -d, --depth <N>      搜索深度 [default: 6]
16      -t, --time <MS>      时间限制 (ms)
17      -e, --engine <NAME> 引擎选择 [default: alphabeta]
```

4.2 UCI 协议 (uci.rs)

实现标准 UCI 协议子集：

- `uci` —— 引擎标识
- `isready` / `readyok` —— 就绪检查
- `position [fen <fen> | startpos] [moves ...]` —— 设置局面
- `go [depth <n> | movetime <ms>]` —— 开始搜索
- `stop` —— 停止搜索
- `quit` —— 退出

五、chess-gui 设计（后续阶段）

5.1 BoardView trait

```
1  pub trait BoardView {
2      /// 返回所有棋子的位置和类型
3      fn pieces(&self, position: &Position) -> Vec<Square, Piece>;
4
5      /// 返回上次走法（用于高亮）
6      fn last_move(&self) -> Option<Move>;
7
8      /// 合法走法标记格
9      fn legal_move_squares(&self, from: Square) -> Vec<Square>;
```

GUI 通过 `BoardView` 获得渲染所需信息，完全隔离内部实现。

六、测试策略

6.1 单元测试层级

层级	测试内容	目标覆盖率
L1	Square, BitBoard, Piece 基础类型	100%
L2	Board 操作	100%
L3	Move 编解码	100%
L4	走法生成（每种棋子）	95%+
L5	特殊规则（易位/过路兵/升变）	100%
L6	FEN 解析/输出往返	100%
L7	Game API	90%+

6.2 棋规测试用例

必须通过的测试：

```
1  #[test]
2  fn initial_position_20_moves();      // 起始局面 20 个合法走法
3  #[test]
4  fn castling_through_check_blocked(); // 王经过被攻击格不能易位
5  #[test]
6  fn en_passant_capture();             // 吃过路兵
7  #[test]
8  fn en_passant_blocked();             // 过路兵被阻挡
9  #[test]
10 fn promotion_all_types();            // 四种升变
11 #[test]
12 fn check_detection();                // 将军检测
13 #[test]
14 fn checkmate_detection();            // 将杀检测
15 #[test]
16 fn stalemate_detection();            // 逼和检测
17 #[test]
18 fn castling_rights_update();         // 易位权限更新
```

6.3 Perft 测试

```
1  #[test]
2  fn perft_startpos() {
3      let pos = Position::startpos();
4      assert_eq!(perft(&mut pos, 1), 20);
```

```

5     assert_eq!(perft(&mut pos, 2), 400);
6     assert_eq!(perft(&mut pos, 3), 8902);
7     assert_eq!(perft(&mut pos, 4), 197281);
8 }
9
10 #[test]
11 fn perft_kiwipete() {
12     // Kiwipete 局面的 perft 值
13     let pos = Position::from_fen(
14         "r3k2r/p1ppqpb1/bn2pnp1/3PN3/1p2P3/2N2Q1p/PPPBPPPP/R3K2R w KQkq - 0
15         1"
16     ).unwrap();
17     assert_eq!(perft(&mut pos, 1), 48);
18     assert_eq!(perft(&mut pos, 2), 2039);
19     assert_eq!(perft(&mut pos, 3), 97862);
20 }

```

6.4 集成测试

- `tests/integration_test.rs` —— 完整对局模拟
- `tests/perft_suite.rs` —— 标准 perft 套件 (10+ 个已知局面)

七、开发阶段规划

Phase 1: 基础类型

交付物:

- `square.rs` —— Square 类型 + 所有常量
- `color.rs` —— Color 枚举
- `piece.rs` —— PieceKind, Piece
- `bitboard.rs` —— BitBoard + 运算符 + 迭代器
- `board.rs` —— Board (12 bitboard + by_square 数组)
- `error.rs` —— ChessError

Phase 2: 局状态与走法

交付物:

- `mv.rs` —— 压缩 Move + MoveFlag
- `castling.rs` —— CastlingRights bitflags
- `zobrist.rs` —— Zobrist 哈希表
- `fen.rs` —— FEN 解析/生成
- `position.rs` —— Position (组装以上所有)

Phase 3: 攻击表与走法生成

交付物:

- `attack.rs` —— 预计算表 + `is_square_attacked`
- `movegen/mod.rs` —— MoveGenerator trait + LegalMoveGenerator
- `movegen/pawn.rs` —— 兵走法生成
- `movegen/knight.rs` —— 马走法生成
- `movegen/bishop.rs` —— 象走法生成
- `movegen/rook.rs` —— 车走法生成
- `movegen/queen.rs` —— 后走法生成
- `movegen/king.rs` —— 王走法生成

Phase 4: 合法性检查

交付物:

- `legality.rs` —— `is_legal` + `is_attacked`
- 走法生成器集成合法性过滤

Phase 5: 走法执行与特殊规则

交付物:

- `makemove.rs` —— MoveMaker + Undo

Phase 6: Game API + Perft

交付物:

- `game.rs` —— Game 高级 API
- `perft.rs` —— Perft 实现

Phase 7: AI 引擎

交付物:

- `chess-ai` crate 结构
- `ChessEngine` trait
- `RandomEngine`
- `AlphaBetaEngine` (基础 Alpha-Beta + 静态搜索)
- `eval.rs` (基础评估函数)

Phase 8: 优化

计划项:

- Magic Bitboard (替换经典射线算法)
- 置换表 (Transposition Table)
- 走法排序优化

- 迭代加深
- 空着裁剪
- Killer/History 启发
- NNUUE 集成点预留
- Criterion benchmarks
- SIMD 加速 (可选)

八、性能设计决策

8.1 零分配走法生成

- 走法生成器使用固定大小的栈数组 (最大容量 256)
- 返回 `Vec<Move>` 用于 API 简洁性, 内部路径使用 `ArrayVec`
- 后续可改为回调模式 `fn generate(&self, pos: &Position, f: impl FnMut(Move))` 避免分配

8.2 增量更新

数据	更新策略
Zobrist Key	XOR 增量: 每次 make/unmake 只 XOR 变化的部分
Halfmove Clock	吃子/兵移动时归零, 否则+1
Fullmove Number	黑方移动后+1
CastlingRights	按掩码清除受影响的权限

8.3 缓存友好的数据结构

- `Board::by_square: [Option<Piece>; 64]` 连续内存, 一次缓存行加载
- `Position` 大小控制在 2 个缓存行以内 (~128 bytes)
- `Undo` 大小控制在 1 个缓存行 (~64 bytes)

九、API 可见性规则

9.1 chess-core 导出策略

`lib.rs` 中 re-export:

```

1 // 公共类型
2 pub use square::Square;
3 pub use color::Color;
4 pub use piece::{Piece, PieceKind};
5 pub use bitboard::BitBoard;
6 pub use board::Board;
7 pub use position::Position;
8 pub use mv::{Move, MoveFlag};

```

```
9  pub use castling::CastlingRights;
10 pub use game::Game;
11 pub use error::ChessError;
12 pub use fen::Fen;
13 pub use perft::perft;
14
15 // 内部模块不公开
16 mod attack;
17 mod movegen;
18 mod makemove;
19 mod zobrist;
20 mod legality;
```

9.2 访问控制矩阵

类型/方法	chess-core 内部	chess-ai	chess-gui/cli
Square, Piece, Color	pub	pub	pub
BitBoard	pub	pub(crate)	不可见
Board 内部	pub(crate)	不可见	不可见
Position 只读方法	pub	pub	pub
Position 可变方法	pub(crate)	不可见	不可见
Game	pub	pub	pub
make_move / unmake_move	pub(crate)	不可见	不可见
MoveGenerator	pub(crate)	不可见	不可见

9.3 封装意图

```
1  外界 (GUI/CLI/AI)
2  |
3  └─ Game (唯一可变入口)
4  |   └─ Game::play(Move)
5  |   └─ Game::undo()
6  |   └─ Game::position() -> &Position
7  |
8  └─ Position (只读)
9  |   └─ piece_at(Square) -> Option<Piece>
10 |   └─ side_to_move() -> Color
11 |   └─ legal_moves (via Game)
12 |   └─ FEN 导入/导出
13 |
14 └─ 基础类型 (Square, Piece, Color, Move)
15     └─ 用于显示和交互
```

十、扩展接口预留

10.1 NNUE 接口预留

```
1 // chess-ai/src/eval.rs 中预留
2 pub struct NnueEvaluator {
3     // Phase 8+ 实现
4     _reserved: (),
5 }
6
7 impl NnueEvaluator {
8     pub fn load_weights(path: &str) -> Result<Self, ChessError>;
9     pub fn evaluate(&self, position: &Position) -> i32;
10 }
```

10.2 搜索扩展点

```
1 // search.rs 中预留
2 pub trait SearchExtension {
3     fn on_node_enter(&mut self, position: &Position, depth: u32);
4     fn on_node_exit(&mut self, position: &Position, score: i32);
5     fn prune(&self, position: &Position, depth: u32) -> Option<i32>;
6 }
```

10.3 UCI 扩展

```
1 // UCI 引擎可以注册自定义选项
2 pub trait UciOption {
3     fn name(&self) -> &str;
4     fn set_value(&mut self, value: &str) -> Result<(), ChessError>;
5     fn get_value(&self) -> String;
6 }
```

十一、代码质量规范

11.1 Rust 工具链配置

```
1 # rust-toolchain.toml
2 [toolchain]
3 channel = "stable"
4 components = ["rustfmt", "clippy"]
```

11.2 必须遵守的规则

- **Edition 2024**: 所有 crate 使用 `edition = "2024"`
- **零 unsafe**: 整个项目不使用 `unsafe` 代码 (Phase 8 如需 SIMD 优化时再评估)
- **禁止裸 `usize`**: 所有棋盘位置使用 `Square` 类型
- **禁止 8x8 数组**: 内部表示使用位棋盘, 不兜底到 `[[Option<Piece>; 8]; 8]`

- **文档注释**: 所有公开 API 必须有 `///` 注释
- **clippy clean**: `cargo clippy --all-targets -- -D warnings`
- **rustfmt**: 所有代码通过 `cargo fmt --check`

11.3 命名约定

范畴	约定	示例
类型	PascalCase	<code>AlphaBetaEngine</code>
函数/方法	snake_case	<code>is_square_attacked</code>
常量	SCREAMING_SNAKE_CASE	<code>MAX_PLY</code>
模块	snake_case	<code>movegen</code>
Crate	kebab-case	<code>chess-core</code>

十二、文件清单

chess-core/src/ 文件总览

文件	行数估算	核心职责
<code>lib.rs</code>	~30	Re-export + crate 文档
<code>square.rs</code>	~150	Square 类型 + 64 个常量 + Display
<code>color.rs</code>	~30	Color 枚举 + flip
<code>piece.rs</code>	~60	PieceKind + Piece
<code>bitboard.rs</code>	~200	BitBoard + 运算 + 迭代器
<code>board.rs</code>	~150	Board: 12 bitboard + by_square
<code>position.rs</code>	~120	Position 结构 + 只读 API
<code>mv.rs</code>	~200	Move(u32) 压缩 + MoveFlag
<code>castling.rs</code>	~30	CastlingRights bitflags
<code>zobrist.rs</code>	~80	Zobrist 键表
<code>fen.rs</code>	~120	FEN 解析/生成
<code>attack.rs</code>	~150	攻击表 + is_square_attacked
<code>movegen/mod.rs</code>	~80	MoveGenerator trait + 聚合
<code>movegen/pawn.rs</code>	~120	兵走法生成
<code>movegen/knight.rs</code>	~40	马走法生成

文件	行数估算	核心职责
<code>movegen/bishop.rs</code>	~50	象走法生成
<code>movegen/rook.rs</code>	~50	车走法生成
<code>movegen/queen.rs</code>	~20	后走法生成
<code>movegen/king.rs</code>	~80	王走法生成 + 易位
<code>legality.rs</code>	~60	is_legal 过滤
<code>makemove.rs</code>	~200	make/unmake + Undo
<code>game.rs</code>	~150	Game 高级 API
<code>error.rs</code>	~40	ChessError
<code>perft.rs</code>	~40	perft 实现
总计	~2260	

十三、验证方案

端到端验证流程

```
1  # 1. 编译整个 workspace
2  cargo build --workspace
3
4  # 2. 运行所有测试
5  cargo test --workspace
6
7  # 3. 代码格式检查
8  cargo fmt --all -- --check
9
10 # 4. Clippy 检查
11 cargo clippy --workspace --all-targets -- -D warnings
12
13 # 5. 运行 perft 基准
14 cargo run -p chess-cli -- perft --depth 4
15
16 # 6. 运行性能基准
17 cargo bench -p benches
18
19 # 7. 交互式测试
20 cargo run -p chess-cli -- play --depth 4
```

持续集成检查项

- `cargo test --workspace` 全部通过
- `cargo fmt --check` 通过
- `cargo clippy -- -D warnings` 通过

- `cargo doc --no-deps` 无警告
- `Perft(4)` 值正确
- 零 `unsafe` 代码块